The attribute name for this is `indexes`. It takes a list of `models.Index` objects, with each object instance specifying the participating fields, as shown in the following example:

```
class VenueStaff(models.Model):
    # first_name, last_name, employee ID number fields

    class Meta:
        indexes = [models.Index(fields=["last_name", "first_name"])]
```

NOTE Remember, indexes make a big performance difference on query results for the fields being searched. Without an index, the database needs to scan the entire table until it finds a matching result. The index works like a hash table, providing a shortcut to finding your values.

5.4.3 Uniqueness

The Django ORM automatically creates the ID field for you as a unique identifier for an object in the table. There may be situations where you want to ensure one or more other fields are also unique within a table. The `unique_together` attribute allows you to do just that.

This attribute takes a list of lists. The nested list contains the fields that are to be unique when combined, which are all in a list, as you can specify more than one set of fields. As a shortcut, you can specify this as a single list if you only have one set of fields that are to be unique together.

Consider the `Room` object, which defines a room in a `Venue`, discussed in chapter 4. You don't want two rooms to have the same name in the same venue. You can avoid this by making the `Room` `name` and the `venue` `ForeignKey` unique together:

```
class Room(models.Model):
    # name and venue field declaration

    class Meta:
        unique_together = [["name", "venue"]]

        # alternate short-cut version as there is only one "uniqueness"
        # unique_together = ["name", "venue"]
```

The enforcement of the `unique_together` clause is done at the database level. If you create an object that violates this in code, an exception will be raised. If you attempt to do the same in the Django Admin, you will get an error message. If you make these kinds of changes to your models, don't forget you'll need to run the `makemigrations` and `migrate` management commands to have them take effect.

More recent versions of Django have a general-purpose mechanisms for specifying constraints on your tables. Django has constraint classes—one for uniqueness and the other for enforcing values. There is also an abstract base class, so you can write your own. The `unique_together` attribute duplicates the constraint case. It hasn't been deprecated yet, but the documentation indicates it may be in the future. Writing your